

Interface vs. Implementation

The `SimpleButton` and `SimpleText` examples bring up the important topic of interface versus implementation. As mentioned in Chapter 4, the interface refers to how something is used, while the implementation refers to how something works (internally).

In an OOP environment, the interface is the set of methods in a class and their related parameters—also known as the *application programming interface (API)*. The implementation is the actual code of all the methods in the class.

An external package such as pygame will most likely come with documentation of the API that explains the calls that are available and the arguments you are expected to pass with each call. The full pygame API documentation is available at <https://www.pygame.org/docs/>.

When you write code that makes calls to pygame, you don't need to worry about the implementation of the methods you are using. For example, when you make a call to `blit()` to draw image, you really don't care *how* `blit()` does what it does; you just need to know *what* the call does and what arguments need to be passed in. On the other side, you can trust that the implementer(s) who wrote the `blit()` method have thought extensively about how to make `blit()` work most efficiently.

In the programming world, we often wear two hats as both the implementer and the application developer, so we need to make an effort to design APIs that not only make sense in the current situation, but also are general enough to be used by future programs of our own and by programs written by other people. Our `SimpleButton` and `SimpleText` classes are good examples, as they are written in a general way so that they can be reused easily. I'll talk more about interface versus implementation in Chapter 8, when we look at encapsulation.

Callbacks

When using a `SimpleButton` object, we handle checking for and reacting to a button click like this: